EXPERIMENT NO. 06 - Simulation of a Baseband Wireless Modulation Scheme



Name: Dibyanshu Shekhar Dey
Roll Number: 25EC64R23

Group Number : 5
Group Member(s):
Akash Kumar (22EC38001)
Dibyanshu Shekhar Dey (25EC64R23)

Design and Simulation Laboratory-Communication Systems
Department of Electronics & Electrical Communication Engineering
Indian Institute of Technology Kharagpur

TITLE: Simulation of a Baseband Wireless Modulation Scheme.

PROBLEM STATEMENTS: :

1. Generate random bit sequences and encode using BPSK and QPSK modulation.
2. Synthesize transmit waveforms for chosen symbol time using both rectangular and raised-cosine pulses
3. Simulate transmission through three types of channels:
   (a) Standard Gaussian channel (fixed gain, no ISI)
   (b) Gaussian channel with ISI (3-tap discrete filter)
   (c) Rayleigh fading wireless channel (non-frequency-selective)

KEY COMPONENTS: MATLAB 25.1.0.2973910 (R2025a) Update 1.

BRIEF THEORY & BACKGROUND:

INTRODUCTION: In a digital communication system, the information source, channel, and end user are usually predefined. The designer's task is to build the transmitter and receiver so that the transmitted message is accurately reproduced at the receiver output according to user requirements and the system remains cost-effective [1].
A typical digital communication system consists of three main pairs of functional blocks: [1]

1. Source encoder–decoder: Removes redundancy from the message signal to use the channel efficiently.
2. Channel encoder–decoder: Adds controlled redundancy to protect against channel noise and errors.
3. Modulator–demodulator: Converts digital symbols into analog waveforms suitable for transmission and recovers them at the receiver.

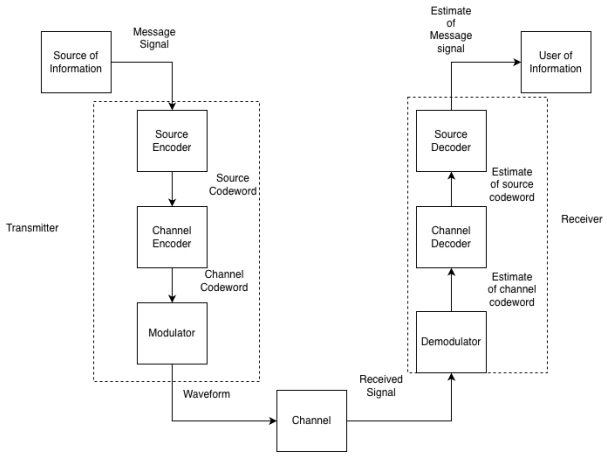


Figure 1: The overall Structure of a Digital Communication system [1].

BPSK: In Binary Phase-Shift Keying (BPSK), digital data bits (1 and 0) are represented by two sinusoidal signals that are identical except for a phase difference of radians (180°). These two signals are antipodal, meaning one is the negative of the other. BPSK is effectively a special case of double sideband suppressed-carrier (DSB-SC) modulation, where the carrier amplitude is constant but its phase shifts by depending on the input bit [2].

The transmitted signal for BPSK is:

$$s_i(t) = \begin{cases} \sqrt{\frac{2E_b}{T_b}} \cos(2\pi f_c t), & \text{for bit 1} \\ -\sqrt{\frac{2E_b}{T_b}} \cos(2\pi f_c t), & \text{for bit 0} \end{cases}$$

where:

- E_b = energy per bit
- T_b = bit duration
- f_c = carrier frequency
- $s_1(t)$ and $s_2(t)$ differ by a phase shift of π radians.

QPSK: An important goal in digital communication is the efficient utilization of channel bandwidth. This is achieved by using bandwidth-conserving modulation schemes such as Quadrature Phase-Shift Keying (QPSK), where information is conveyed through the phase of a sinusoidal carrier [2].
In QPSK, the carrier phase can take one of four equally spaced values, for example, $\pi/4$, $3\pi/4$, $5\pi/4$, and $7\pi/4$. Thus, the transmitted signal is defined as

$$s_i(t) = \begin{cases} \sqrt{\frac{2E}{T}} \cos\left(2\pi f_c t + (2i-1)\frac{\pi}{4}\right), & 0 \le t \le T, \\ 0, & \text{elsewhere,} \end{cases}$$

where $i = 1, 2, 3, 4$.

Here:

- E is the transmitted signal energy per symbol.
- T is the symbol duration.
- f_c is the carrier frequency.

Each of the four phase values corresponds to a unique pair of bits called a *dibit*. A common Gray-coded mapping for these dibits is: 10, 00, 01, and 11. Since each symbol represents two bits, the symbol duration is twice the bit duration:

$$T = 2T_b.$$

Using trigonometric identities, the transmitted signal can also be written as:

$$s_i(t) = \sqrt{\frac{2E}{T}} \cos\left((2i-1)\frac{\pi}{4}\right) \cos(2\pi f_c t) - \sqrt{\frac{2E}{T}} \sin\left((2i-1)\frac{\pi}{4}\right) \sin(2\pi f_c t),$$

showing that QPSK can be viewed as the combination of two orthogonal BPSK signals transmitted in quadrature (i.e., 90° out of phase).
Baseband Modulation:

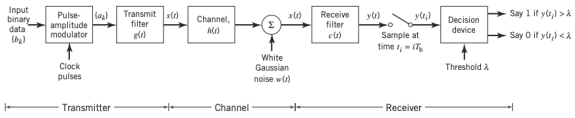


Figure 2: Baseband Wireless Modulation.

Baseband Wireless Modulation: The diagram represents a **baseband digital communication system** consisting of three main sections [1]: the *transmitter*, the *channel*, and the *receiver*.

- **Transmitter:** The input binary data sequence $\{b_k\}$ is converted into amplitude symbols $\{a_k\}$ using a pulse-amplitude modulator. These symbols are then shaped by a transmit filter $g(t)$ to produce the transmitted waveform $s(t)$.
- **Channel:** The signal passes through a channel characterized by an impulse response $h(t)$ and is corrupted by additive white Gaussian noise $w(t)$, resulting in the received signal $x(t)$.
- **Receiver:** The received signal is filtered by a receive filter $c(t)$ (often matched to $g(t)$) to produce $y(t)$, which is then sampled at optimal instants $t_i = iT_b$. A decision device compares the sample $y(t_i)$ with threshold λ to decide whether a binary '1' or '0' was transmitted.

AWGN: AWGN is a basic noise model used in communication systems where [3]:
The received signal $r(t)$ equals the transmitted signal $s(t)$ plus a noise term $w(t)$:

$$r(t) = s(t) + w(t)$$

The term *"additive"* means the noise simply adds to the signal. The term *"white"* means the noise has a flat *power spectral density (PSD)* across frequency (uniform across the band). The term *"Gaussian"* means the noise samples follow a *Gaussian (normal) distribution* with zero mean.
The noise PSD is often given by $N_0/2$ (in W/Hz) for real-valued noise. If the system bandwidth is B, then the noise power P_w in that band is:

$$P_w = N_0 \times B$$

or in a sampled system with sampling rate F_s:

$$P_w = N_0 \cdot \frac{F_s}{2}$$

AWGN is used because it yields tractable closed-form expressions for error probability,and many systems (e.g., point-to-point microwave links) approximate this scenario if channel impairments beyond noise are minimal.
Rayleigh Fading: Rayleigh fading is a wireless channel phenomenon that occurs when there is *no direct line-of-sight (NLOS)* path between the transmitter and receiver. The received

signal results from the superposition of multiple reflected, scattered, and diffracted components arriving with different phases [4].
The channel coefficient h is modeled as a complex Gaussian random variable with zero mean and equal variance in its real and imaginary parts. Consequently, the magnitude $|h|$ follows a *Rayleigh distribution*. The received signal is given by:

$$y(t) = h \cdot x(t) + n(t)$$

where $x(t)$ is the transmitted signal, h represents the multipath fading coefficient, and $n(t)$ denotes additive white Gaussian noise (AWGN). Because the amplitude of h can experience deep fades, Rayleigh fading causes more severe performance degradation (higher bit error rate) compared to an AWGN channel. Diversity techniques and channel estimation are typically employed to mitigate its effects.

Rayleigh Probability Density Function (PDF):

$$f_A(a) = \frac{a}{\sigma^2} \exp\left(-\frac{a^2}{2\sigma^2}\right), \quad a \geq 0$$

where σ^2 is the variance of the Gaussian components of h.

Rectangular Pulse: A rectangular pulse is the simplest form of digital pulse used in communication systems. It has a constant amplitude A for a fixed duration T. In digital signaling, the presence or absence of such a pulse can represent binary symbols "1" and "0." A continuous sequence of these pulses, spaced by T seconds, defines a bit or symbol rate of $1/T$ pulses per second. The spectrum of a rectangular pulse, obtained through its Fourier transform, has a sinc-shaped form, with nulls occurring at multiples of the symbol rate $f_0 = 1/T$. Although simple to generate, the rectangular pulse theoretically occupies infinite bandwidth, so practical systems use pulse-shaping filters to limit bandwidth and reduce intersymbol interference (ISI).
Mathematical Representation:

$$p(t) = \begin{cases} A, & -\frac{T}{2} \leq t \leq \frac{T}{2}, \\ 0, & \text{otherwise.} \end{cases}$$

$$P(f) = AT\,\mathrm{sinc}(fT) = AT\,\frac{\sin(\pi f T)}{\pi f T}$$

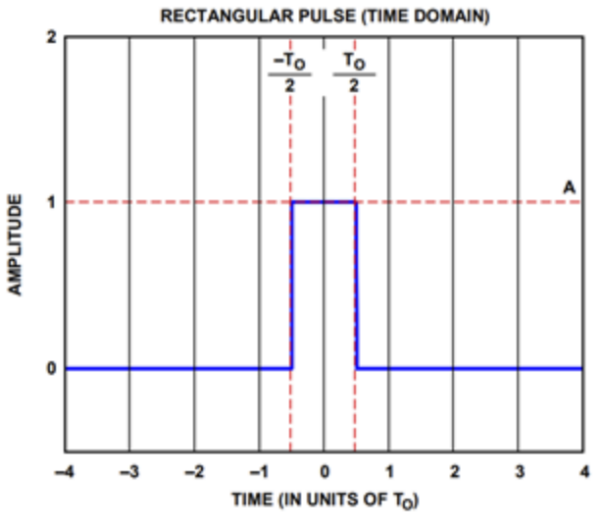


Figure 3: Time Domain Raised Rectangular Pulse Response (Normalized) [5].

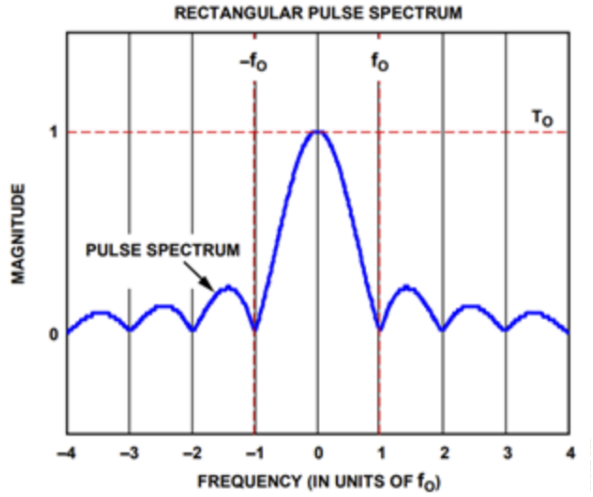


Figure 4: Frequency Domain Raised Rectangular Pulse Response (Normalized) [5].

Raised Cosine Pulse : The *raised cosine pulse* is a practical pulse-shaping filter used in digital communication systems to satisfy the *Nyquist criterion* for zero intersymbol interference (ISI). It provides a smooth transition in the frequency domain to limit bandwidth while ensuring that adjacent symbols do not interfere with each other.

The raised cosine spectrum is characterized by a *roll-off factor* α (where $0 \leq \alpha \leq 1$), which determines the excess bandwidth beyond the ideal Nyquist bandwidth $W_b = 1/(2T)$. A higher roll-off factor α produces a smoother transition in the frequency domain but increases the bandwidth requirement, whereas a smaller α results in a sharper transition and greater delay in the time domain. The raised cosine frequency response is flat within its main lobe and rolls off gradually according to a cosine function near the band edges. Its time-domain representation decays faster (proportional to $1/t^3$) than the sinc function (proportional to $1/t$), making it more practical for real-world communication systems that must limit bandwidth and avoid abrupt transitions [6].

Frequency-Domain Raised Cosine Spectrum:

$$\hat{g}_\alpha(f) = \begin{cases} T, & 0 \le |f| \le \dfrac{1-\alpha}{2T}, \\ T\cos^2\left[\dfrac{\pi T}{2\alpha}\left(|f| - \dfrac{1-\alpha}{2T}\right)\right], & \dfrac{1-\alpha}{2T} \le |f| \le \dfrac{1+\alpha}{2T}, \\ 0, & |f| \ge \dfrac{1+\alpha}{2T}. \end{cases}$$

Time-Domain Raised Cosine Pulse:

$$g_\alpha(t) = \frac{\sin(\pi t/T)}{\pi t/T}\,\frac{\cos(\pi\alpha t/T)}{1-4\alpha^2 t^2/T^2}$$

Interpretation:

- $\hat{g}_\alpha(f)$ — Raised cosine spectrum in the frequency domain.
- $g_\alpha(t)$ — Time-domain raised cosine pulse.
- α — Roll-off factor controlling the excess bandwidth and smoothness.
- T — Symbol duration.

This pulse ensures *zero intersymbol interference (ISI)*, controlled bandwidth, and faster decay than the sinc pulse, making it ideal for both baseband and passband digital communication systems [6].

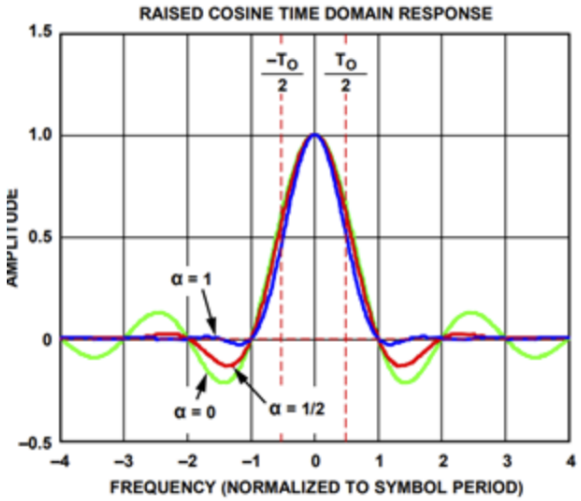


Figure 5: Time Domain Raised Cosine Pulse Response (Normalized) [5].

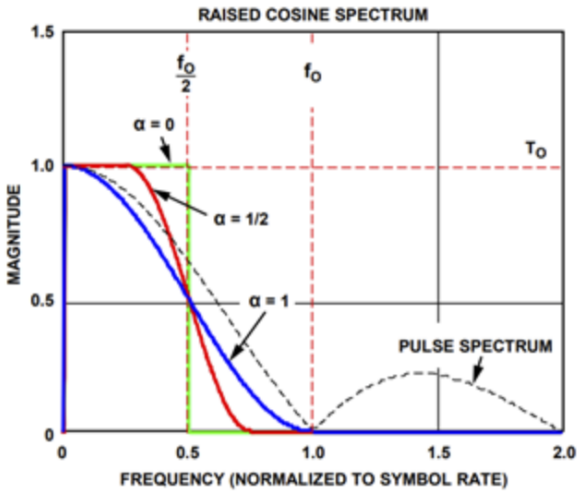


Figure 6: Frequency Domain Raised Cosine Pulse Response (Normalized) [5].

OUTCOMES:

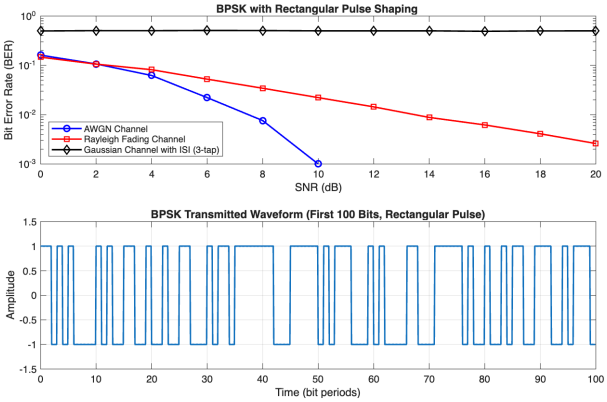


Figure 7: BER for BPSK Rectangular Pulse Signal for three channels along with the Transmitted data.

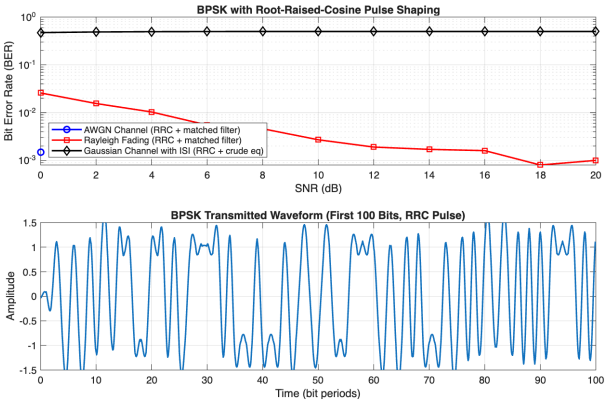


Figure 8: BER for BPSK Root Raised Cosine Pulse Signal for three channels along with the Transmitted data.

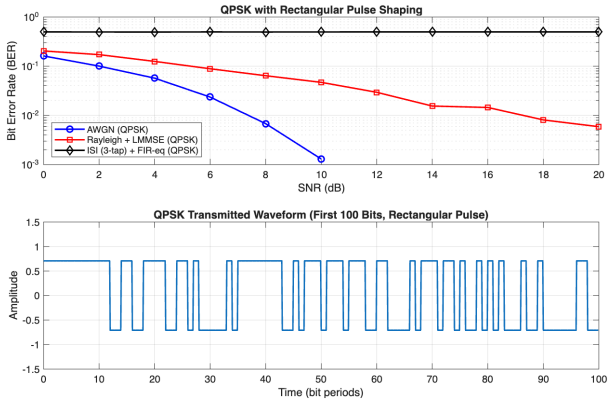


Figure 9: BER for QPSK Rectangular Pulse Signal for three channels along with the Transmitted data.

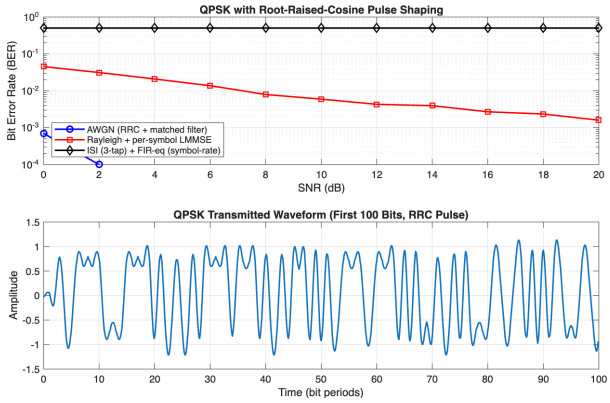


Figure 10: BER for QPSK Root Raised Cosine Pulse Signal for three channels along with the Transmitted data.

DISCUSSIONS:
AWGN Channel:

- Represents an ideal communication environment with only random Gaussian noise.
- Bit Error Rate (BER) decreases **exponentially** with increasing Signal-to-Noise Ratio (SNR).
- Theoretical QPSK BER expression:

$$\text{BER} = Q\left(\sqrt{\frac{2E_b}{N_0}}\right)$$

- At high SNR, BER approaches zero, showing that system performance is limited only by random noise.

Gaussian Channel with ISI (3-Tap Filter):

- Inter-Symbol Interference (ISI) causes overlap between adjacent symbols.

- Symbol spreading distorts symbol boundaries, making detection less reliable.
- Even at high SNR, BER does not reach zero due to persistent ISI effects.
- Performance can only improve with equalization or better pulse shaping techniques.

Rayleigh Fading Channel:

- Models wireless multipath environments with no direct line-of-sight component.
- Received signals experience amplitude and phase variations due to constructive and destructive interference.
- BER performance degrades significantly compared to the AWGN channel.
- At low-to-moderate SNR levels, deep fades cause high error rates despite strong signal power.

Comparative Observations:

- AWGN Channel: Noise-limited — BER decreases rapidly with increasing SNR.
- ISI Channel: Bandwidth-limited — BER saturates even at high SNR due to symbol overlap.
- Rayleigh Channel: Fading-limited — BER remains high at low-to-moderate SNR because of multipath fading.
- Overall, channel impairments such as ISI and fading affect system performance more severely than random Gaussian noise.

ACKNOWLEDGEMENTS:
I would like to thank TAs M V Srivatsav and E Maansi for their help in the experiment. I would like to extend my thanks to Prof Amitalok J Budkuley, for explaining the concepts. I would like to thank my lab partner Akash Kumar, with whom I was able to perform some of the prerequisites in the lab. I would like to thank my colleagues Saptarshi Bhattacharjee, Maytank Singh and Soumyajit Adak, with whom the late night sessions have turned into coding together, discussion of the concepts involved and more understanding of the material.

References

[1] S. Haykin, *Digital Communication Systems*. Hoboken, NJ: Wiley, 2013, ISBN: 978-1-118-29512-9.

[2] S. Haykin and M. Moher, *Introduction to Analog and Digital Communications*, 2nd. Hoboken, NJ: John Wiley & Sons, Inc., 2006, ISBN: 978-0-471-64735-1.

[3] W. Pi, *"additive white gaussian noise (awgn)"*, Online article, Accessed: 2025-10-30, 2015. [Online]. Available: https://wirelesspi.com/additive-white-gaussian-noise-awgn/.

[4] SalimWireless, *"difference between awgn and rayleigh fading"*, Online, Accessed: 2025-10-30, Nov. 2022. [Online]. Available: https://www.salimwireless.com/2022/11/difference-between-awgn-and-rayleigh.html.

[5] K. Gentile, "An-922: Digital pulse-shaping filter basics," Analog Devices, Inc., Application Note AN-922, 2002. [Online]. Available: https://www.analog.com/en/resources/app-notes/an-922.html.

[6] R. G. Gallager, *Principles of Digital Communication*. Cambridge, UK: Cambridge University Press, 2007, ISBN: 978-0-521-86756-5.

APPENDIX:
MATLAB Code:
BPSK Waveform generation using Rectangular Pulse and passing it through three channels and getting the BER.

```
    clc; clear; close all;

%% PARAMETERS
N = 1e4;                % Number of bits
fs = 10;                % Samples per bit (rectangular pulse samples per
    bit)
bits = randi([0 1], 1, N);
bpsk = 2*bits - 1;        % BPSK symbols (+1/-1)
snr_dB = 0:2:20;          % SNR range
ber_awgn = zeros(size(snr_dB));
ber_rayleigh = zeros(size(snr_dB));
ber_isi = zeros(size(snr_dB));   % For Gaussian ISI channel
ber_rayleigh_zf = zeros(size(snr_dB));

%% RECTANGULAR PULSE SHAPING
pulse = ones(1, fs);                        % Rectangular pulse shape
tx_waveform = kron(bpsk, pulse);            % Upsample and repeat each symbol
tx_waveform = tx_waveform / sqrt(mean(abs(tx_waveform).^2));  % Normalize
    power

% Restore symbol-rate representation for BER loop
bpsk = tx_waveform(1:fs:end);   % Take 1 sample per symbol (symbol-spaced
    version)

%% 3-TAP GAUSSIAN ISI CHANNEL
h_isi = [0.9 0.3 0.2];          % Example discrete-time ISI channel taps
h_isi = h_isi / norm(h_isi);    % Normalize energy to 1

%% SIMULATION
for i = 1:length(snr_dB)
    snr_lin = 10^(snr_dB(i)/10);
    sigma2 = 1 / snr_lin;
    % --- AWGN Channel ---
    rx_awgn = awgn(bpsk, snr_dB(i), 'measured');   % Received signal
    rx_eq_1 = (1 / (1 + sigma2)) * rx_awgn;          % LMMSE equalization
    rx_bits_eq = rx_eq_1 > 0;                        % Detection
    ber_awgn_lmmse(i) = sum(rx_bits_eq ~= bits) / N;


    % --- Rayleigh Fading Channel ---
    h = (randn(1,N) + 1j*randn(1,N)) / sqrt(2);   % Rayleigh fading coeffs
    rx_rayleigh = h .* bpsk;                       % Apply fading
    rx_rayleigh = awgn(rx_rayleigh, snr_dB(i), 'measured');
    %LMMSE
    rx_eq = real((conj(h) ./ (abs(h).^2 + sigma2)) .* rx_rayleigh);  %
        Equalization LMMSE
```

```
    rx_bits_rayleigh = rx_eq > 0;  % Detection
    ber_rayleigh(i) = sum(rx_bits_rayleigh ~= bits) / N;

    % --- Gaussian Channel with ISI (3-tap discrete filter) ---
    rx_isi = conv(bpsk, h_isi, 'same');           % Apply ISI channel
    rx_isi = awgn(rx_isi, snr_dB(i), 'measured'); % Add Gaussian noise
    rx_eq_3 = real((conj(h) ./ (abs(h).^2 + sigma2)) .* rx_isi);  %
       Equalization LMMSE
    rx_bits_isi = rx_eq_3 > 0;                  % Detection
    ber_isi(i) = sum(rx_bits_isi ~= bits) / N;

end


%% SELECT FIRST 100 BITS FOR VISUALIZATION
num_bits_plot = 100;
samples_to_plot = num_bits_plot * fs;
waveform_100 = tx_waveform(1:samples_to_plot);

%% TIME AXIS
t = (0:length(waveform_100)-1) / fs;     % Time in symbol periods

%% PLOT RESULTS
figure;
subplot(2,1,1);
semilogy(snr_dB, ber_awgn_lmmse, 'bo-', 'LineWidth', 1.5); hold on;
semilogy(snr_dB, ber_rayleigh, 'rs-', 'LineWidth', 1.5); hold on;
semilogy(snr_dB, ber_isi, 'kd-', 'LineWidth', 1.5);
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
legend('AWGN Channel', 'Rayleigh Fading Channel', 'Gaussian Channel with
   ISI (3-tap)', 'Location', 'southwest');
title('BPSK with Rectangular Pulse Shaping ');


%% TIME AXIS
t = (0:length(waveform_100)-1) / fs;     % Time in symbol periods
subplot(2,1,2);
%% PLOT WAVEFORM
plot(t, waveform_100, 'LineWidth', 1.5);
grid on;
xlabel('Time (bit periods)');
ylabel('Amplitude');
title('BPSK Transmitted Waveform (First 100 Bits, Rectangular Pulse)');
ylim([-1.5 1.5]);
```

BPSK Waveform generation using Raised Cosine Pulse and passing it through three channels and getting the BER.

```
    clc; clear; close all;

%% PARAMETERS
N = 1e4;                % Number of symbols (bits for BPSK)
sps = 10;               % samples per symbol (previously fs)
bits = randi([0 1], 1, N);
symbols = 2*bits - 1;   % BPSK symbols (+1/-1)
snr_dB = 0:2:20;        % SNR range

```

```
ber_awgn_lmmse = zeros(size(snr_dB));
ber_rayleigh = zeros(size(snr_dB));
ber_isi = zeros(size(snr_dB));

%% RRC (root-raised-cosine) PULSE SHAPING
rolloff = 0.25;          % roll-off factor (0..1)
span = 6;                % filter span in symbols
% rcosdesign
tx_pulse = rcosdesign(rolloff, span, sps, 'sqrt');
% normalize to unit energy
tx_pulse = tx_pulse / sqrt(sum(tx_pulse.^2));

% Upsample + pulse shape
tx_waveform = upfirdn(symbols, tx_pulse, sps, 1);

% normalize transmitted average power to 1
tx_waveform = tx_waveform / sqrt(mean(abs(tx_waveform).^2));
rx_mf = tx_pulse;  % matched filter

%% 3-TAP GAUSSIAN ISI CHANNEL (symbol-rate discrete taps)
h_isi = [0.9 0.3 0.2];
h_isi = h_isi / norm(h_isi);

%% SIMULATION
for ii = 1:length(snr_dB)
    snr_lin = 10^(snr_dB(ii)/10);
    sigma2 = 1 / snr_lin;

    %% --- AWGN Channel (no multipath) ---
    % TX: tx_waveform
    rx_awgn_wave = awgn(tx_waveform, snr_dB(ii), 'measured');   % add AWGN
    % Matched filter
    rx_mf_out = conv(rx_awgn_wave, rx_mf, 'same');
    % account for group delay of pulse
    delay = (length(tx_pulse)-1)/2;
    % sample indexes
    first_sample = round(delay) + 1;
    sampled = rx_mf_out(first_sample : sps : first_sample + sps*(N-1));
    % normalize sampled to unit average power
    sampled = sampled / sqrt(mean(abs(sampled).^2));
    % LMMSE equalization for AWGN
    rx_eq_1 = (1 / (1 + sigma2)) * real(sampled);
    rx_bits_eq = rx_eq_1 > 0;
    ber_awgn_lmmse(ii) = sum(rx_bits_eq ~= bits) / N;

    %% --- Rayleigh Fading Channel (fixed) ---
    %Rayleigh fading coefficients
    h_sym = (randn(1,N) + 1j*randn(1,N)) / sqrt(2);   % length N

    % pulse-shape
    symbols_faded = h_sym .* symbols;
    tx_waveform_faded = upfirdn(symbols_faded, tx_pulse, sps, 1);
    tx_waveform_faded = tx_waveform_faded / sqrt(mean(abs(tx_waveform_faded
        ).^2)); % normalize

    % Pass through AWGN
    rx_faded_wave = awgn(tx_waveform_faded, snr_dB(ii), 'measured');
    % Matched filter
```

```
    rx_mf_out = conv(rx_faded_wave, rx_mf, 'same');
    first_sample = round((length(tx_pulse)-1)/2) + 1;
    sampled = rx_mf_out(first_sample : sps : first_sample + sps*(N-1));
    % LMMSE per-symbol scalar equalizer:
    rx_eq = real((conj(h_sym) ./ (abs(h_sym).^2 + sigma2)) .* sampled);
    rx_bits_rayleigh = rx_eq > 0;
    ber_rayleigh(ii) = sum(rx_bits_rayleigh ~= bits) / N;

    %% --- Gaussian Channel with ISI (3-tap discrete filter at symbol-rate) ---
    symbols_after_isi = conv(symbols, h_isi, 'same');   % symbol-rate ISI applied
    tx_waveform_isi = upfirdn(symbols_after_isi, tx_pulse, sps, 1);
    tx_waveform_isi = tx_waveform_isi / sqrt(mean(abs(tx_waveform_isi).^2));
    rx_isi_wave = awgn(tx_waveform_isi, snr_dB(ii), 'measured');
    % matched filter + sample
    rx_mf_out = conv(rx_isi_wave, rx_mf, 'same');
    sampled_isi = rx_mf_out(first_sample : sps : first_sample + sps*(N-1));
    ch_energy = sum(abs(h_isi).^2);  % total channel energy
    rx_eq_3 = real((1 / (ch_energy + sigma2)) * sampled_isi);
    rx_bits_isi = rx_eq_3 > 0;
    ber_isi(ii) = sum(rx_bits_isi ~= bits) / N;

end

%% SELECT FIRST 100 BITS FOR VISUALIZATION
num_bits_plot = 100;
samples_to_plot = num_bits_plot * sps;
waveform_100 = tx_waveform(1:samples_to_plot);

%% TIME AXIS
t = (0:length(waveform_100)-1) / sps;     % Time in symbol periods

%% PLOT RESULTS
figure;
subplot(2,1,1);
semilogy(snr_dB, ber_awgn_lmmse, 'bo-', 'LineWidth', 1.5); hold on;
semilogy(snr_dB, ber_rayleigh, 'rs-', 'LineWidth', 1.5);
semilogy(snr_dB, ber_isi, 'kd-', 'LineWidth', 1.5);
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
legend('AWGN Channel (RRC + matched filter)', 'Rayleigh Fading (RRC + matched filter)', 'Gaussian Channel with ISI (RRC + crude eq)', 'Location', 'southwest');
title('BPSK with Root-Raised-Cosine Pulse Shaping');
grid on;

%% PLOT WAVEFORM
subplot(2,1,2);
plot(t, waveform_100, 'LineWidth', 1.5);
grid on;
xlabel('Time (bit periods)');
ylabel('Amplitude');
title('BPSK Transmitted Waveform (First 100 Bits, RRC Pulse)');
ylim([-1.5 1.5]);
```

QPSK Waveform generation using Rectangular Pulse and passing it through three channels and

getting the BER.

```
    clc; clear; close all;

%% PARAMETERS
Nbits = 1e4;                    % number of bits (make even for QPSK)
if mod(Nbits,2)~=0, Nbits = Nbits+1; end
Nsym = Nbits/2;                 % number of QPSK symbols
fs = 10;                        % samples per symbol (rectangular)
bits = randi([0 1], 1, Nbits);

snr_dB = 0:2:20;
ber_awgn = zeros(size(snr_dB));
ber_rayleigh = zeros(size(snr_dB));
ber_isi = zeros(size(snr_dB));

%% QPSK MAPPING (Gray-like)
bI = bits(1:2:end);
bQ = bits(2:2:end);
I = 1 - 2*bI;
Q = 1 - 2*bQ;
symbols = (I + 1j*Q) / sqrt(2);

%% RECTANGULAR PULSE SHAPING (symbol-rate -> waveform)
pulse = ones(1, fs);
tx_waveform = kron(symbols, pulse);
tx_waveform = tx_waveform / sqrt(mean(abs(tx_waveform).^2));
symbols = tx_waveform(1:fs:end);

%% 3-TAP GAUSSIAN ISI CHANNEL (real taps)
h_isi = [0.9 0.3 0.2];
h_isi = h_isi / norm(h_isi);

% Equalizer (ZF-ish) for h_isi
Lh = length(h_isi);
Lw = 7;             % equalizer length
% Toeplitz convolution matrix
conv_len = Lh + Lw - 1;
% Toeplitz
Hmat = zeros(conv_len, Lw);
for k = 1:Lw
    Hmat(k:k+Lh-1, k) = h_isi(:);
end
% target impuls
center = ceil(conv_len/2);
d = zeros(conv_len,1);
d(center) = 1;
% Equalizer
w_eq = pinv(Hmat) * d;   % Lw x 1

%% SIMULATION
for idx = 1:length(snr_dB)
    snr = snr_dB(idx);
    snr_lin = 10^(snr/10);
    sigma2 = 1 / snr_lin;

    % --- AWGN channel (no channel) ---
    rx_awgn = awgn(symbols, snr, 'measured');
```

```
    % symbol detection: decision on real and imag separately
    decI = real(rx_awgn) > 0;
    decQ = imag(rx_awgn) > 0;
    rx_bits = zeros(1, Nbits);
    rx_bits(1:2:end) = ~decI;
    rx_bits(2:2:end) = ~decQ;
    ber_awgn(idx) = sum(rx_bits ~= bits) / Nbits;

    % --- Rayleigh fading (per-symbol) ---
    h = (randn(1, Nsym) + 1j*randn(1, Nsym)) / sqrt(2);  % complex fading
        per symbol
    rx_ray = h .* symbols;                                % apply fading
    rx_ray = awgn(rx_ray, snr, 'measured');               % add AWGN
    % Per-symbol LMMSE equalizer: s_hat = conj(h)/( |h|^2 + sigma2 ) * y
    s_hat = (conj(h) ./ (abs(h).^2 + sigma2)) .* rx_ray;
    % Detect
    decI = real(s_hat) > 0;
    decQ = imag(s_hat) > 0;
    rx_bits = zeros(1, Nbits);
    rx_bits(1:2:end) = ~decI;
    rx_bits(2:2:end) = ~decQ;
    ber_rayleigh(idx) = sum(rx_bits ~= bits) / Nbits;

    % --- Gaussian channel with ISI (3-tap) ---
    rx_isi = conv(symbols, h_isi, 'same');    % symbol-rate ISI
    rx_isi = awgn(rx_isi, snr, 'measured');  % add noise
    % Equalizer w_eq (FIR)
    y_eq = filter(w_eq, 1, rx_isi);
    % account for equalizer+channel delay
    % delay of center sample relative to input first sample:
    total_delay = center - 1;   % zero-based index;
    % extract symbol-spaced estimates
    start_idx = total_delay + 1;
    end_idx = start_idx + Nsym - 1;
    % pad y_eq
    if end_idx > length(y_eq)
        y_eq = [y_eq, zeros(1, end_idx - length(y_eq))];
    end
    s_hat_isi = y_eq(start_idx:end_idx);
    % detect QPSK
    decI = real(s_hat_isi) > 0;
    decQ = imag(s_hat_isi) > 0;
    rx_bits = zeros(1, Nbits);
    rx_bits(1:2:end) = ~decI;
    rx_bits(2:2:end) = ~decQ;
    ber_isi(idx) = sum(rx_bits ~= bits) / Nbits;
end

%% SELECT FIRST 100 BITS FOR VISUALIZATION
num_bits_plot = 100;
samples_to_plot = num_bits_plot * fs;
waveform_100 = tx_waveform(1:samples_to_plot);

%% TIME AXIS
t = (0:length(waveform_100)-1) / fs;     % Time in symbol periods

%% PLOT
figure;
```

```
subplot(2,1,1);
semilogy(snr_dB, ber_awgn, 'bo-', 'LineWidth', 1.5); hold on;
semilogy(snr_dB, ber_rayleigh, 'rs-', 'LineWidth', 1.5);
semilogy(snr_dB, ber_isi, 'kd-', 'LineWidth', 1.5);
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
legend('AWGN (QPSK)', 'Rayleigh + LMMSE (QPSK)', 'ISI (3-tap) + FIR-eq (
    QPSK)', 'Location', 'southwest');
title('QPSK with Rectangular Pulse Shaping');

%% PLOT WAVEFORM
subplot(2,1,2);
plot(t, waveform_100, 'LineWidth', 1.5);
grid on;
xlabel('Time (bit periods)');
ylabel('Amplitude');
title('QPSK Transmitted Waveform (First 100 Bits, Rectangular Pulse)');
ylim([-1.5 1.5]);
```

QPSK Waveform generation using Raised Cosine Pulse and passing it through three channels and getting the BER.

```
    clc; clear; close all;

%% PARAMETERS
N = 1e4;                % Number of QPSK symbols
sps = 10;               % samples per symbol
Nbits = 2 * N;          % total bits
bits = randi([0 1], 1, Nbits);

% make SNR vector
snr_dB = 0:2:20;        % SNR range
ber_awgn_lmmse = zeros(size(snr_dB));
ber_rayleigh = zeros(size(snr_dB));
ber_isi = zeros(size(snr_dB));

%% RRC (root-raised-cosine) PULSE SHAPING
rolloff = 0.25;         % roll-off factor
span = 6;               % fspan in symbols
tx_pulse = rcosdesign(rolloff, span, sps, 'sqrt');  % RRC pulse (sqrt-RRC)
tx_pulse = tx_pulse / sqrt(sum(tx_pulse.^2));      % normalize

% matched filter (same as tx for sqrt-RRC)
rx_mf = tx_pulse;

%% QPSK MAPPING (Gray-like)
bI = bits(1:2:end);   % bits for I
bQ = bits(2:2:end);   % bits for Q
I = 1 - 2 * bI;
Q = 1 - 2 * bQ;
symbols = (I + 1j * Q) / sqrt(2);   % unit-energy QPSK symbols

%% 3-TAP GAUSSIAN ISI CHANNEL (symbol-rate)
h_isi = [0.9 0.3 0.2];
h_isi = h_isi / norm(h_isi);   % normalize energy

% Equalizer (ZF-style) for symbol-rate h_isi
```

```
Lh = length(h_isi);
Lw = 7;  % equalizer length
conv_len = Lh + Lw - 1;
Hmat = zeros(conv_len, Lw);
for k = 1:Lw
    Hmat(k:k+Lh-1, k) = h_isi(:);
end
center = ceil(conv_len/2);
d = zeros(conv_len,1); d(center) = 1;
w_eq = pinv(Hmat) * d;   % symbol-rate equalizer taps

%% TRANSMIT: pulse shaping (symbol -> waveform)
tx_waveform = upfirdn(symbols, tx_pulse, sps, 1);
tx_waveform = tx_waveform / sqrt(mean(abs(tx_waveform).^2));  % normalize
    avg power

% matched filter group delay (in samples of waveform)
mf_delay = (length(tx_pulse)-1)/2;      % in waveform samples

%% SIMULATION
for ii = 1:length(snr_dB)
    snr_lin = 10^(snr_dB(ii)/10);
    sigma2 = 1 / snr_lin;   % noise variance per complex dimension (unit-
        energy symbols)

    %% --- AWGN channel (no multipath) ---
    rx_awgn_wave = awgn(tx_waveform, snr_dB(ii), 'measured');
    rx_mf_out = conv(rx_awgn_wave, rx_mf, 'same');
    first_sample = round(mf_delay) + 1;
    idx = first_sample : sps : (first_sample + sps*(N-1));
    sampled = rx_mf_out(idx);   % symbol-spaced complex samples
    sampled = sampled / sqrt(mean(abs(sampled).^2)); % optional normalize

    % simple scalar LMMSE for AWGN (no channel) - apply to complex samples
    rx_eq_awgn = (1 / (1 + sigma2)) * sampled;
    % Decision: I and Q separately
    decI = real(rx_eq_awgn) > 0;
    decQ = imag(rx_eq_awgn) > 0;
    rx_bits = zeros(1, Nbits);
    rx_bits(1:2:end) = ~decI;   % mapping back to bits (0->+1)
    rx_bits(2:2:end) = ~decQ;
    ber_awgn_lmmse(ii) = sum(rx_bits ~= bits) / Nbits;

    %% --- Rayleigh fading channel (per-symbol) ---
    h_sym = (randn(1,N) + 1j*randn(1,N)) / sqrt(2);   % complex fading per
        symbol
    symbols_faded = h_sym .* symbols;                % apply fading at
        symbol-rate
    tx_waveform_faded = upfirdn(symbols_faded, tx_pulse, sps, 1);
    tx_waveform_faded = tx_waveform_faded / sqrt(mean(abs(tx_waveform_faded
        ).^2));

    rx_faded_wave = awgn(tx_waveform_faded, snr_dB(ii), 'measured');
    rx_mf_out = conv(rx_faded_wave, rx_mf, 'same');
    sampled = rx_mf_out(idx);   % symbol-spaced received complex samples (
        length N)

    % Per-symbol complex LMMSE equalizer using known h_sym
```

```
    eq_coeff = conj(h_sym) ./ (abs(h_sym).^2 + sigma2);
    s_hat = eq_coeff .* sampled;
    decI = real(s_hat) > 0;
    decQ = imag(s_hat) > 0;
    rx_bits = zeros(1, Nbits);
    rx_bits(1:2:end) = ~decI;
    rx_bits(2:2:end) = ~decQ;
    ber_rayleigh(ii) = sum(rx_bits ~= bits) / Nbits;

    %% --- Gaussian channel with ISI (symbol-rate) ---
    % Apply symbol-rate ISI first, then pulse-shape
    symbols_after_isi = conv(symbols, h_isi, 'same');   % length N
    tx_waveform_isi = upfirdn(symbols_after_isi, tx_pulse, sps, 1);
    tx_waveform_isi = tx_waveform_isi / sqrt(mean(abs(tx_waveform_isi).^2))
        ;

    rx_isi_wave = awgn(tx_waveform_isi, snr_dB(ii), 'measured');
    rx_mf_out = conv(rx_isi_wave, rx_mf, 'same');
    sampled_isi = rx_mf_out(idx);    % symbol-spaced samples (length N)

    % AEqualizer (w_eq) to sampled_isi
    y_eq = filter(w_eq, 1, sampled_isi);
    % Equalizer delay: center-1
    eq_delay = center - 1;
    start_idx = eq_delay + 1;
    end_idx = start_idx + N - 1;
    % pad if needed
    if end_idx > length(y_eq)
        y_eq = [y_eq, zeros(1, end_idx - length(y_eq))];
    end
    s_hat_isi = y_eq(start_idx:end_idx);

    decI = real(s_hat_isi) > 0;
    decQ = imag(s_hat_isi) > 0;
    rx_bits = zeros(1, Nbits);
    rx_bits(1:2:end) = ~decI;
    rx_bits(2:2:end) = ~decQ;
    ber_isi(ii) = sum(rx_bits ~= bits) / Nbits;

end

%% SELECT FIRST 100 BITS FOR VISUALIZATION
num_bits_plot = 100;
samples_to_plot = num_bits_plot * sps;
waveform_100 = tx_waveform(1:samples_to_plot);

%% TIME AXIS
t = (0:length(waveform_100)-1) / sps;     % Time in symbol periods

%% PLOT RESULTS
figure;
subplot(2,1,1);
semilogy(snr_dB, ber_awgn_lmmse, 'bo-', 'LineWidth', 1.5); hold on;
semilogy(snr_dB, ber_rayleigh, 'rs-', 'LineWidth', 1.5);
semilogy(snr_dB, ber_isi, 'kd-', 'LineWidth', 1.5);
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
```

```
145 legend('AWGN␣(RRC␣+␣matched␣filter)', 'Rayleigh␣+␣per-symbol␣LMMSE', 'ISI␣
        (3-tap)␣+␣FIR-eq␣(symbol-rate)', 'Location', 'southwest');
146 title('QPSK␣with␣Root-Raised-Cosine␣Pulse␣Shaping');
147
148 %% PLOT WAVEFORM
149 subplot(2,1,2);
150 plot(t, waveform_100, 'LineWidth', 1.5);
151 grid on;
152 xlabel('Time␣(bit␣periods)');
153 ylabel('Amplitude');
154 title('QPSK␣Transmitted␣Waveform␣(First␣100␣Bits,␣RRC␣Pulse)');
155 ylim([-1.5 1.5]);
```